

A Hash-Based Trust Framework for Multi-Agent AI Systems:

Ensuring Instruction Integrity Through Cryptographic Verification

Dharmesh Kothari
AI Research Practitioner

ABSTRACT

Multi-agent AI systems, in which a central orchestrator delegates tasks to specialised subagents, are increasingly deployed in enterprise environments to automate complex workflows. However, a critical and largely unaddressed vulnerability exists in the communication layer between orchestrators and subagents: the absence of cryptographic verification means that subagents have no reliable mechanism to confirm that instructions they receive originate from a legitimate orchestrator and have not been tampered with in transit. This paper proposes a lightweight trust verification framework using SHA-256 cryptographic hashing combined with asymmetric digital signatures to address this vulnerability. The proposed framework enables subagents to independently verify instruction authenticity and integrity before execution, without requiring blockchain infrastructure, distributed consensus mechanisms, or significant computational overhead. The framework is designed to be architecture-agnostic and implementable within existing enterprise AI deployments. We describe the framework architecture, verification protocol, threat model coverage, implementation considerations, and limitations. The proposed approach offers a practical, low-cost path to verifiable agent-to-agent trust in production multi-agent AI systems.

Keywords: *multi-agent systems, AI governance, cryptographic hashing, digital signatures, instruction integrity, orchestrator, subagent, trust verification, agent security, enterprise AI*

1. Introduction

The deployment of multi-agent AI systems — architectures in which a central orchestrating agent decomposes a high-level goal and delegates subtasks to specialised subagents — has grown significantly as organisations seek to automate complex, multi-step workflows. In these systems, the orchestrator acts as a coordinator, issuing instructions to subagents that in turn interact with external tools, data sources, and systems through mechanisms such as the Model Context Protocol (MCP) [1]. Subagents execute instructions and return results, which the orchestrator aggregates to produce a final output.

Despite the increasing sophistication of these systems, a fundamental security vulnerability persists in the instruction channel between orchestrators and subagents. In the majority of current implementations, subagents execute instructions on the basis of trust alone — there is no cryptographic mechanism by which a subagent can verify that an instruction genuinely originated from the legitimate orchestrator, that the instruction has not been modified in transit, or that the instruction is not a replay of a previously issued command. This gap creates meaningful attack surfaces, particularly in the context of prompt injection attacks, man-in-the-middle interception, and rogue agent impersonation.

Blockchain-based approaches have been proposed as a solution to this verification problem, offering immutability, decentralised consensus, and cryptographic audit trails [2]. However, blockchain introduces substantial complexity, infrastructure requirements, latency, and cost that make it impractical for most enterprise deployments. This paper argues that the core security properties required for multi-agent instruction verification — authenticity, integrity, non-repudiation, and replay prevention — can be achieved through a significantly simpler mechanism: cryptographic hashing combined with asymmetric digital signatures.

The primary contributions of this paper are as follows:

- A formal description of the instruction integrity problem in multi-agent AI systems
- A threat model covering the principal attack vectors in agent-to-agent communication
- A lightweight hash-based trust verification framework for orchestrator-to-subagent instruction channels
- A verification protocol that subagents can implement independently of the orchestrator
- An analysis of framework limitations and directions for future work

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 describes the problem in detail. Section 4 presents the threat model. Section 5 introduces the proposed framework. Section 6 describes the verification protocol. Section 7 analyses security properties. Section 8 discusses implementation considerations. Section 9 addresses limitations. Section 10 concludes.

2. Related Work

Research on multi-agent AI systems has primarily focused on coordination strategies, task decomposition, and performance optimisation [3,4]. Security considerations in agent-to-agent communication have received comparatively less attention, with most work addressing the external boundary — securing the interface between the AI system and the user — rather than the internal channels between agents.

Prompt injection attacks, in which malicious content embedded in external data attempts to redirect agent behaviour, have been studied by Greshake et al. [5], who demonstrated that agents

processing untrusted content are vulnerable to instruction hijacking. This work is directly relevant to the subagent context, where instructions may arrive through channels that mix legitimate orchestrator output with externally sourced content.

Blockchain-based approaches to multi-agent coordination have been proposed in several contexts. Calvaresi et al. [6] examine the intersection of blockchain and multi-agent systems, identifying the potential for immutable audit trails and verifiable coordination. However, these approaches introduce consensus overhead and infrastructure complexity that limit their applicability in latency-sensitive enterprise deployments.

Digital signature schemes, including RSA [7] and elliptic curve variants such as ECDSA [8], are well-established mechanisms for message authentication and integrity verification. Their application to AI agent communication channels has not, to our knowledge, been formally proposed as a standalone governance framework prior to this work.

The Model Context Protocol (MCP) [1], introduced by Anthropic in 2024, provides a standardised interface for connecting AI agents to external tools and data sources. While MCP defines the communication structure, it does not currently specify cryptographic verification of instructions passed through the protocol. The framework proposed in this paper is designed to complement MCP and similar agent communication protocols.

3. Problem Statement

3.1 Multi-Agent System Architecture

A multi-agent AI system as considered in this paper consists of the following components: an orchestrator agent that receives a high-level goal, decomposes it into subtasks, and delegates those subtasks to one or more subagents; subagents that receive instructions from the orchestrator, execute them using available tools, and return results; tools and external systems accessible to subagents through defined interfaces; and a communication channel through which instructions and results are passed between the orchestrator and subagents.

3.2 The Instruction Integrity Problem

In current implementations, the instruction channel between orchestrator and subagent is typically unverified. A subagent receiving an instruction cannot distinguish between an instruction that originated legitimately from the orchestrator and one that was:

- Injected by malicious content in data the agent is processing
- Intercepted and modified by an adversary in the communication channel
- Fabricated by another agent impersonating the orchestrator
- Replayed from a previously legitimate instruction to cause unintended re-execution

This vulnerability is particularly significant when subagents have access to tools that can take consequential actions — writing to databases, sending communications, modifying files, or interacting with external APIs. An instruction that bypasses verification and reaches tool execution can cause data corruption, unauthorised disclosure, or system disruption.

3.3 Why Existing Approaches Are Insufficient

Network-level security measures such as TLS encryption protect the confidentiality and integrity of data in transit but do not address the problem of instruction origin verification — a message can be encrypted in transit and still originate from an unauthorised source. Application-level authentication mechanisms such as API keys provide coarse-grained access control but do not enable per-instruction verification of origin and content integrity. Blockchain approaches provide strong guarantees but introduce infrastructure, latency, and cost barriers that are prohibitive for most enterprise deployments.

4. Threat Model

We consider the following threat model for multi-agent AI systems operating in enterprise environments.

4.1 Assumed Capabilities of the Adversary

The adversary is assumed to be capable of: injecting content into data sources that agents process; intercepting and modifying messages in the communication channel between agents; impersonating the orchestrator by sending fabricated instructions to subagents; and replaying previously captured legitimate instructions to cause re-execution of actions.

The adversary is assumed not to have access to the orchestrator's private cryptographic key. Compromise of the private key is treated as outside the threat model and addressed separately in Section 9.

4.2 Attack Vectors

Attack Vector	Description	Potential Impact
Prompt Injection	Malicious instructions embedded in external data processed by agents	Unauthorised tool execution, data exfiltration
Man-in-the-Middle	Interception and modification of instructions in the communication channel	Instruction tampering, action redirection
Orchestrator Impersonation	Fabricated instructions claiming to originate from the orchestrator	Unauthorised agent actions

Replay Attack	Re-submission of previously legitimate instructions	Duplicate or unintended re-execution of actions
Result Fabrication	Subagent returning false results to the orchestrator	Incorrect decisions based on fabricated data

Table 1. Principal attack vectors in multi-agent AI communication channels.

5. Proposed Framework

5.1 Framework Overview

The proposed framework introduces cryptographic verification into the orchestrator-to-subagent instruction channel using two well-established cryptographic primitives: SHA-256 hashing for instruction integrity verification, and asymmetric digital signatures (RSA-2048 or ECDSA P-256) for instruction authenticity verification. The framework requires no blockchain infrastructure, no distributed consensus, and no changes to the underlying AI model or tool interfaces. It operates as a verification layer that wraps existing instruction channels.

5.2 Cryptographic Components

SHA-256 Hashing

SHA-256 is a cryptographic hash function that produces a 256-bit digest of arbitrary input data [9]. Its properties relevant to this framework are: determinism — the same input always produces the same hash; collision resistance — it is computationally infeasible to find two different inputs that produce the same hash; and avalanche effect — any modification to the input produces a completely different hash. These properties make SHA-256 suitable for instruction integrity verification: if an instruction is modified in any way after hashing, the hash will not match when recalculated by the subagent.

Asymmetric Digital Signatures

Asymmetric cryptography uses a key pair consisting of a private key, held exclusively by the orchestrator, and a public key, distributed to all subagents. The orchestrator signs the instruction hash using its private key to produce a digital signature. Subagents verify the signature using the orchestrator's public key. A valid signature proves that the instruction was signed by the holder of the private key and that the signed content has not been modified since signing [7].

5.3 Instruction Envelope Structure

Every instruction issued by the orchestrator is packaged in a verified instruction envelope containing the following fields:

Field	Type	Description
-------	------	-------------

instruction_id	UUID	Unique identifier for this instruction instance
orchestrator_id	String	Identifier of the issuing orchestrator
target_agent_id	String	Identifier of the intended recipient subagent
payload	JSON	The instruction content and parameters
timestamp	ISO 8601	Time of instruction creation
ttl	Integer	Time-to-live in seconds — prevents replay attacks
payload_hash	Hex String	SHA-256 hash of the payload field
envelope_hash	Hex String	SHA-256 hash of all fields except signature
signature	Base64	Digital signature of envelope_hash using orchestrator private key

Table 2. Verified instruction envelope structure.

5.4 Key Management

At system initialisation, the orchestrator generates an asymmetric key pair. The private key is stored securely in the orchestrator environment and never transmitted. The public key is distributed to all authorised subagents through a secure out-of-band channel at deployment time. Subagents store the orchestrator public key in their local configuration and use it for signature verification. Key rotation should be performed on a defined schedule, with a transition period during which both the old and new public keys are accepted by subagents to prevent service disruption.

6. Verification Protocol

6.1 Orchestrator — Instruction Creation

The orchestrator follows these steps when creating a verified instruction:

- Step 1: Construct the instruction payload as a structured JSON object containing the task specification and parameters
- Step 2: Compute `payload_hash = SHA256(JSON.stringify(payload))`
- Step 3: Construct the envelope with all fields populated including `payload_hash` and a current timestamp
- Step 4: Compute `envelope_hash = SHA256(JSON.stringify(envelope excluding signature field))`
- Step 5: Compute `signature = sign(private_key, envelope_hash)` using the selected signature algorithm
- Step 6: Attach the signature to the envelope
- Step 7: Transmit the complete envelope to the target subagent

- Step 8: Record the `instruction_id` and `envelope_hash` in the local audit log

6.2 Subagent — Instruction Verification

Upon receiving an instruction envelope, the subagent performs the following verification steps before any execution:

- Step 1: Extract the `target_agent_id` field and confirm it matches the subagent's own identifier — reject if not addressed to this agent
- Step 2: Extract the `timestamp` and `ttd` fields — reject if current time exceeds `timestamp + ttd`, preventing replay attacks
- Step 3: Check `instruction_id` against a local cache of recently processed instruction IDs — reject if already processed, preventing duplicate execution
- Step 4: Recompute the `envelope_hash` from all envelope fields except the signature field
- Step 5: Verify the signature field using the orchestrator's public key and the recomputed `envelope_hash` — reject if signature verification fails
- Step 6: Recompute `payload_hash` from the `payload` field and compare with the `payload_hash` field in the envelope — reject if hashes do not match
- Step 7: If all checks pass, proceed to execution — if any check fails, reject the instruction, log the rejection with the failure reason, and raise an alert

6.3 Result Verification

The same mechanism applies in reverse for result transmission from subagent to orchestrator. Each subagent has its own key pair. Subagents sign their results using their private key. The orchestrator verifies each result using the corresponding subagent's public key before incorporating it into downstream processing. This prevents result fabrication by compromised or rogue subagents.

6.4 Audit Logging

All verification events — both successful and failed — are written to an append-only audit log. Each log entry records the `instruction_id`, the verification outcome, the timestamp, and in the case of failure, the specific check that failed. The audit log itself is protected against modification through the use of a hash chain: each log entry includes the hash of the previous entry, forming a linked sequence that detects any retrospective alteration of log records.

7. Security Analysis

We analyse the framework's coverage of the threat model described in Section 4.

Attack Vector	Framework Response	Coverage
---------------	--------------------	----------

Prompt Injection	Injected content cannot carry a valid orchestrator signature. Signature verification fails at Step 5. Instruction rejected.	Full
Man-in-the-Middle	Any modification to the envelope invalidates the envelope_hash and signature. Detected at Steps 4-5. Instruction rejected.	Full
Orchestrator Impersonation	Without access to the orchestrator private key, a fabricated instruction cannot produce a valid signature. Rejected at Step 5.	Full
Replay Attack	TTL check at Step 2 and instruction_id deduplication at Step 3 prevent re-execution of captured instructions.	Full
Result Fabrication	Subagent result signing and orchestrator-side verification prevent acceptance of unsigned or incorrectly signed results.	Full
Private Key Compromise	Framework does not address this case. Key compromise requires key revocation and rotation procedures outside the framework scope.	Partial

Table 3. Security analysis against the threat model.

The framework provides full coverage of four of the five principal attack vectors identified in the threat model. Private key compromise, the one partial coverage case, requires organisational key management procedures rather than technical controls within the framework itself. This is consistent with standard cryptographic system design, where the security of the entire scheme depends on the security of the private key.

8. Implementation Considerations

8.1 Computational Overhead

SHA-256 hashing and RSA or ECDSA signature operations are computationally inexpensive relative to the inference cost of large language model calls. In representative benchmarks, SHA-256 hashing of a 1KB instruction payload completes in under one millisecond on modern hardware. RSA-2048 signature verification completes in approximately 0.5 milliseconds. ECDSA P-256 verification is approximately ten times faster. The total overhead introduced by framework verification is therefore negligible relative to the latency of LLM inference, which typically ranges from hundreds of milliseconds to several seconds.

8.2 Audit Log Storage

The audit log can be implemented using several approaches depending on organisational requirements and existing infrastructure. For enterprise deployments requiring strong tamper

evidence, AWS Quantum Ledger Database (QLDB) or Azure Immutable Storage provide managed immutable ledger capabilities without blockchain infrastructure. For simpler deployments, an append-only log file with a hash chain provides a practical and low-cost alternative. Version control systems such as Git can also serve as audit log stores, as every commit is cryptographically linked to its predecessors.

8.3 Integration with Existing Systems

The framework is designed as a wrapper layer around existing instruction channels and does not require modification of the underlying AI model, tool interfaces, or orchestration logic. Implementation requires: generating key pairs for the orchestrator and each subagent at deployment; distributing public keys to relevant agents through a secure channel; wrapping outbound instructions in verified envelopes at the orchestrator; and adding a verification step at the beginning of each subagent's instruction processing pipeline. Reference implementation libraries for SHA-256 and digital signatures are available in all major programming languages, including Python (cryptography library), Node.js (built-in crypto module), and Java (java.security package).

8.4 Recommended Algorithms

Component	Recommended Algorithm	Alternative
Hashing	SHA-256	SHA-3-256
Digital Signature	ECDSA P-256	RSA-2048
Key Exchange	ECDH P-256	RSA-2048 encryption
Instruction ID	UUID v4	CSPRNG-generated 128-bit value

Table 4. Recommended cryptographic algorithms.

9. Limitations and Future Work

9.1 Private Key Security

The security of the entire framework depends on the secrecy of the orchestrator's private key. If the private key is compromised, an adversary can issue arbitrary instructions that will pass all verification checks. Mitigation requires hardware security modules (HSMs) for key storage, strict access controls on the orchestrator environment, and key rotation procedures with defined revocation mechanisms. These are organisational and infrastructure requirements beyond the scope of the framework itself.

9.2 Subagent Compromise

The framework verifies that instructions come from the legitimate orchestrator but does not address the case where a subagent itself is compromised and executes instructions correctly while returning falsified results. Result signing by subagents (Section 6.3) mitigates result fabrication but does not prevent a compromised subagent from taking unauthorised side actions during execution. Runtime monitoring of subagent tool call behaviour is a complementary control.

9.3 Instruction Confidentiality

The framework addresses instruction integrity and authenticity but does not provide instruction confidentiality. Instruction payloads are transmitted in cleartext within the envelope. Organisations with confidentiality requirements should combine the framework with transport-layer encryption or payload encryption using the subagent's public key.

9.4 Future Work

Several directions for future work are identified:

- Formalisation of the verification protocol using process algebra or model checking to provide mathematical security proofs
- Extension of the framework to peer-to-peer subagent communication in architectures where subagents interact directly
- Development of a standardised open specification to enable interoperability across different multi-agent frameworks
- Empirical evaluation of framework performance at scale in production multi-agent deployments
- Integration with emerging agent communication standards and protocols

10. Conclusion

This paper has identified a significant and largely unaddressed vulnerability in multi-agent AI systems: the absence of cryptographic verification in the instruction channel between orchestrators and subagents. Without verification, subagents have no reliable mechanism to confirm that instructions are genuine, unmodified, and intended for them — exposing the system to prompt injection, man-in-the-middle attacks, orchestrator impersonation, and replay attacks.

We have proposed a lightweight framework using SHA-256 hashing and asymmetric digital signatures that addresses these vulnerabilities without requiring blockchain infrastructure. The framework introduces a verified instruction envelope structure and a subagent verification protocol that together provide authenticity, integrity, non-repudiation, and replay protection for agent-to-agent instruction channels. Security analysis demonstrates full coverage of the principal attack vectors in the identified threat model.

The framework is designed to be practical: it imposes negligible computational overhead, integrates with existing agent architectures without modification of underlying models or tools, and relies on well-established cryptographic primitives with mature library support across all major platforms.

As multi-agent AI systems become increasingly prevalent in enterprise environments and are granted access to consequential tools and data, the security of agent-to-agent communication channels becomes a foundational requirement for responsible deployment. The framework presented in this paper offers a concrete and immediately implementable step toward verifiable trust in multi-agent AI systems.

References.

- [1] Anthropic. (2024). Model Context Protocol. Anthropic Technical Documentation. <https://modelcontextprotocol.io>
- [2] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>
- [3] Wooldridge, M. (2009). An Introduction to MultiAgent Systems (2nd ed.). John Wiley and Sons.
- [4] Russell, S., and Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- [5] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not What You Have Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections. arXiv:2302.12173.
- [6] Calvaresi, D., Dubovitskaya, A., Retaggi, G., Dubovitskaya, A., Schumacher, M., and Dragoni, N. (2019). Multi-Agent Systems and Blockchain: Results from a Systematic Literature Review. In *Advances in Practical Applications of Survivable Agents and Multi-Agent Systems*. Springer.
- [7] Rivest, R. L., Shamir, A., and Adleman, L. (1978). A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Communications of the ACM*, 21(2), 120–126.
- [8] Johnson, D., Menezes, A., and Vanstone, S. (2001). The Elliptic Curve Digital Signature Algorithm (ECDSA). *International Journal of Information Security*, 1(1), 36–63.
- [9] National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS). FIPS Publication 180-4. U.S. Department of Commerce.
- [10] Shostack, A. (2014). *Threat Modeling: Designing for Security*. John Wiley and Sons.

Manuscript submitted to SSRN. AI Research Practitioner. India